P.Vishwa Nandan(192473015)

Q7 – Design a Concurrent, Fault-Tolerant Processor from First Principles (10 Marks)

1. Aim

To design and implement a concurrent, fault-tolerant Java application using ExecutorService, Callable, Future, and concurrent collections to process 10,000 student result files efficiently.

The system must handle missing or corrupted files without stopping the entire process and generate a final summary of successful and failed records.

2. Problem Description

The examination cell needs to process 10,000 student answer-script score files at the end of a semester.

Each file must be:

1. Read from storage.
2. Validated to ensure the data is correct.
3. Processed to calculate the student's result.
4. Recorded as either a successful or failed operation.

Some files may be missing or corrupted.

The system must process multiple files simultaneously, handle individual failures, and generate a final summary.

3. Stage 1 – Interpret the Problem (Joint)

3.1 Tasks That Can Run in Parallel

The following operations can be performed independently for each student:

- Reading the student's score file.
- Validating the student ID.
- Checking whether the score is valid.
- Calculating the result.
- Recording success or failure.

Since each file contains independent student data, multiple files can be processed simultaneously using multiple threads.

3.2 Shared Data

The application maintains two shared collections:

Successful Results: Stores the student ID, score, and final result of successfully processed files.

Failed Results: Stores the filename and reason for failure of files that could not be processed.

Both collections use ConcurrentLinkedQueue, which supports safe operations from multiple threads.

3.3 Possible Failures and Their Handling

Failure	Reason	Handling
Missing file	Student file does not exist	Record failure
Corrupted file	File contains invalid data	Record failure
Invalid score	Score is not a number	Record failure
Invalid score range	Score is below 0 or above 100	Record failure
Empty file	Required information is missing	Record failure
Unexpected task failure	An unhandled error occurs during processing	Record failure in the main thread

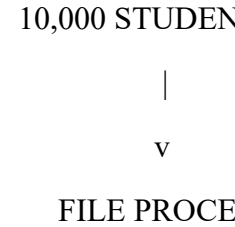
3.4 Measurable Requirements

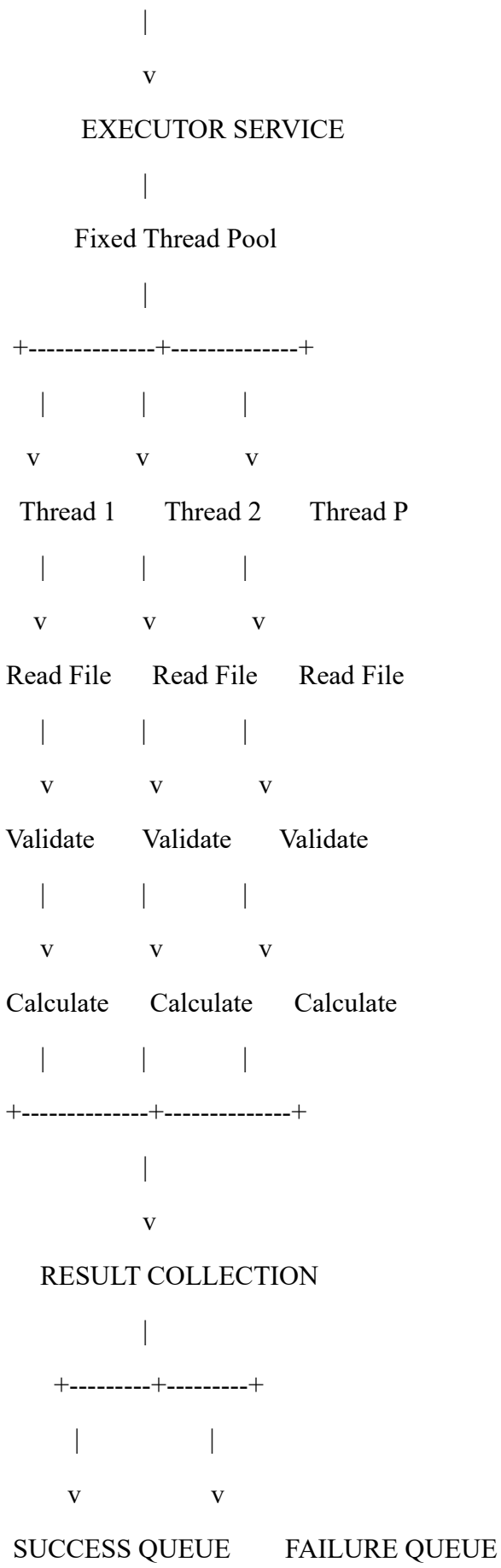
The system must satisfy the following requirements:

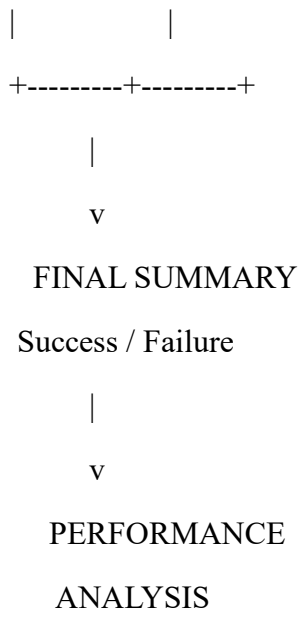
1. Process 10,000 student file tasks.
2. Use a fixed thread pool with 1, 2, 4, and 8 threads.
3. Handle missing and corrupted files separately.
4. Continue processing when an individual file fails.
5. Maintain correct success and failure counts.
6. Generate a final summary after all tasks complete.
7. Ensure that shared result collections are accessed safely.

4. System Architecture

4.1 Architecture Diagram




EXECUTOR SERVICE
Fixed Thread Pool
Thread 1
Thread 2
Thread P
Read File
Read File
Read File
Validate
Validate
Validate
Calculate
Calculate
Calculate
RESULT COLLECTION
SUCCESS QUEUE
FAILURE QUEUE



4.2 Components Used

ExecutorService: Creates and manages a fixed number of worker threads.

Callable: Represents a task that can be executed by a thread and can return a value.

Future: Represents the result of an asynchronous task. It allows the main program to check task completion and retrieve its result.

ConcurrentLinkedQueue: Stores successful and failed results safely when multiple threads access the collections.

CompletionService: Allows the main program to receive completed tasks without waiting for them in their original submission order.

5. Stage 2 – Design (Student A)

5.1 Design Using ExecutorService, Callable and Future

The application uses a fixed-size thread pool.

For example, with four threads, four student files can be processed concurrently. When one thread finishes, it can process another file.

Each file is processed using a Callable task.

The Future returned by the CompletionService represents the submitted task. The main program uses get() to confirm its completion.

5.2 Class Design

The program contains the following components:

ResultProcessor

- Creates the thread pool.

- Submits student file tasks.
- Collects completed tasks.
- Generates the final summary.
- Measures execution time.

ScoreTask

- Reads one student file.
- Validates the file contents.
- Calculates the result.
- Records successful or failed processing.

StudentResult

- Stores student ID.
- Stores examination score.
- Stores PASS or FAIL status.

FailureRecord

- Stores the filename.
- Stores the reason for failure.

RunSummary

- Stores the success count.
- Stores the failure count.
- Stores execution time.
- Stores sample failure details.

5.3 Pseudocode

START

Create successful result queue

Create failed result queue

Create fixed thread pool

```
FOR every student file
    Submit file-processing task
    Task performs:
        Read file
        IF file is missing
            Record failure
        ELSE IF file contains invalid data
            Record failure
        ELSE
            Validate score
            Calculate result
            Store successful result
        END IF
END FOR
Wait for all submitted tasks
Collect completed Future objects
Calculate successful count
Calculate failed count
```

Display final summary

Display execution time

SHUT DOWN thread pool

STOP

5.4 Why ConcurrentLinkedQueue Is Used

Multiple threads may finish processing at nearly the same time.

If a normal ArrayList is shared without synchronization, concurrent updates may cause inconsistent results.

ConcurrentLinkedQueue supports thread-safe insertion and retrieval.

Therefore, it is suitable for maintaining the shared success and failure collections.

6.2 Program Explanation

Step 1 – Creating Student Files

The program creates a small dataset containing three valid files, one corrupted file, and one missing file.

It also creates 10,000 file tasks for the performance experiment.

Step 2 – Creating Threads

Executors.newFixedThreadPool(poolSize) creates a fixed number of worker threads.

For example:

Executors.newFixedThreadPool(4);

This creates a pool containing four worker threads.

Step 3 – Submitting Tasks

Each student file is submitted as a Callable task through the CompletionService.

Step 4 – Reading and Validating

Every task reads its corresponding file and checks whether the student ID and score are valid.

Step 5 – Handling Failures

Missing files, invalid scores, and corrupted data are recorded in the failure queue.

The exception handling is performed inside each task. Therefore, one bad file does not stop the other tasks.

Step 6 – Recording Results

Valid results are stored in the successful results queue.

Failed records are stored in the failure queue.

Step 7 – Collecting Future Results

The main thread collects completed Future objects and calls get() to confirm that each task has completed.

Step 8 – Generating the Summary

The application calculates the total number of successful and failed files and displays the processing time.

7. Stage 3 – Testing and Verification (Student B)

7.1 Small Dataset Verification

The program first processes five hand-checked file tasks.

File	Condition	Expected Result
Student 1	Valid score 80	Success
Student 2	Valid score 65	Success
Student 3	Valid score 35	Success
Student 4	Invalid score format	Failure
Student 5	Missing file	Failure

Expected Output:

STAGE 1: SMALL DATASET VERIFICATION

Successful files: 3

Failed files: 2

Small dataset verification: PASSED

This confirms that the program correctly separates successful and failed records.

7.2 Large Dataset Verification

The large dataset contains 10,000 file tasks.

The program creates:

- 9,980 valid files.
- 10 corrupted files.
- 10 missing files.

Therefore:

Total files = 10,000

Successful files = 9,980

Failed files = 20

The expected output for every thread-pool size is:

Successful files: 9980

Failed files: 20

7.3 Thread Pool Performance Experiment

The program performs three runs for each thread-pool size and calculates the median execution time.

The following table can be completed using the actual output from the program.

Number of Threads	Successful Files	Failed Files	Median Time
1	9980	20	Record actual time
2	9980	20	Record actual time
4	9980	20	Record actual time
8	9980	20	Record actual time

Important Observation:

The execution time may decrease when the number of threads increases because multiple files can be processed simultaneously.

However, more threads do not always improve performance. Disk speed, CPU capacity, context switching, and thread scheduling also affect execution time.

The actual timings must be recorded after running the program on the computer.

7.4 Failure Verification

The following test cases verify fault tolerance.

Test Case	Input	Expected Behaviour

TC1	Valid student file	Process successfully
TC2	Missing student file	Record failure and continue
TC3	Corrupted file	Record failure and continue
TC4	Text instead of score	Record invalid score
TC5	Score below 0	Record validation failure
TC6	Score above 100	Record validation failure
TC7	Empty file	Record missing information
TC8	Multiple invalid files	Continue processing remaining files

7.5 Verifying an Unhandled Task Failure

The program handles expected file-related failures inside ScoreTask.

The Future.get() call is also protected by an ExecutionException catch block.

If a task encounters an unexpected exception that it does not handle internally, the exception is reported by the Future. The main thread records the failure and continues collecting the remaining completed tasks.

This prevents one unexpected task failure from terminating the entire processing operation.

8. Stage 4 – Defend the Design (Joint)

8.1 Recommended Thread Pool Size

A fixed thread pool is suitable because it controls the maximum number of active worker threads.

The recommended pool size should be selected based on the experimental results.

For this file-processing application, four threads can be a reasonable starting point for testing. However, the final choice should depend on the actual performance measurements.

Using eight threads may improve performance when the storage device can handle parallel file reads. It may also increase contention and scheduling overhead.

Therefore, the fastest measured configuration should be selected rather than automatically choosing the largest pool.

8.2 How Does the System Guarantee a Complete Summary?

The program provides a complete summary through the following mechanisms:

1. Every student file task is submitted to the CompletionService.
2. Each task records its success or expected failure.

3. ConcurrentLinkedQueue safely stores the shared results.
4. The main thread waits for every submitted Future to complete.
5. Unexpected task failures are also recorded.
6. Only after all tasks are collected does the program calculate the final counts.

This ensures that each file task contributes to either the success count or the failure count, provided the process is allowed to complete normally.

8.3 Remaining Limitations

The current design has some limitations:

- Processing 10,000 files creates many small file operations, which can make storage performance the main bottleneck.
- Submitting all tasks together requires memory for Future objects and queued tasks.
- If the application is terminated suddenly, results held only in memory may be lost.
- The program uses a local file system and does not support distributed processing.

For a production application, results should also be saved to a database or durable storage so that processing can be resumed after a failure.

9. Alternative Concept and Complexity Analysis

9.1 Alternative 1 – Parallel Streams

Instead of using ExecutorService, the application can use Java parallel streams.

A collection of student file paths can be processed using:

```
files.parallelStream().forEach(file -> {
   // Read file
   // Validate student data
   // Calculate result
   // Record success or failure
});
```

Explanation:

Parallel streams automatically divide work among multiple worker threads managed by the ForkJoin framework.

The developer does not need to manually create and submit every task.

However, expected failures must still be caught inside the processing operation. Otherwise, an exception may interrupt the stream operation.

9.2 Alternative 2 – Creating One Thread Per File

Another approach is creating a separate Thread object for every student file.

For 10,000 files, this approach would create 10,000 threads.

Example:

```
for (int i = 1; i <= 10000; i++) {

    final int id = i;

    Thread t = new Thread(() -> {

        // Read and process student file

    });

    t.start();
}
```

Disadvantages:

- Creates a very large number of threads.
- Consumes more memory because every thread requires stack space.
- Increases context-switching overhead.
- Can exhaust operating system thread resources.
- Makes thread management difficult.

Therefore, creating one thread per file is not recommended.

9.3 Comparison of the Three Approaches

Feature	ExecutorService	Parallel Streams	One Thread Per File
Thread management	Controlled pool	Managed automatically	Manual
Maximum worker threads	Fixed pool size	Depends on execution environment	Up to 10,000

Task submission	Callable and Future	Stream operations	Thread creation
Failure handling	Per-task handling	Must handle inside stream operation	Must handle in every thread
Resource control	Good	Framework-managed	Poor
Suitability	Recommended	Suitable for simpler parallel operations	Not recommended

9.4 Time Complexity

Let:

- N = Number of student files.
- P = Number of worker threads.
- S = Average time required to read and process one file.

ExecutorService:

The approximate processing time is:

$O(N \times S / P)$

This assumes reasonably balanced tasks and sufficient CPU and storage capacity. Actual performance depends on I/O and scheduling overhead.

Parallel Streams:

The approximate processing time is also:

$O(N \times S / P)$

The framework manages the worker threads automatically.

One Thread Per File:

The total processing work remains:

$O(N \times S)$

Although many threads may run concurrently, creating and scheduling thousands of threads introduces significant overhead.

9.5 Space Complexity

Let P be the number of worker threads and S_t be the approximate stack memory reserved per thread.

ExecutorService:

- Worker-thread stack memory: $O(P \times S_t)$.
- Task and Future storage: O(N).
- Result storage: O(N).

Parallel Streams:

- Worker-thread stack memory: $O(P \times S_t)$.
- Result storage: O(N), depending on the collection strategy.
- No need to create one Future object manually for every file.

One Thread Per File:

- Thread stack memory: $O(N \times S_t)$.
- Thread object storage: O(N).
- Result storage: O(N).

Therefore, the fixed thread pool provides better control over the number of active threads and their associated resource usage.

10. Overall Engineering Decision

The ExecutorService approach is selected for the semester result-processing system.

Reasons:

1. It allows a fixed number of worker threads.
2. It supports Callable and Future for task execution and completion tracking.
3. It handles missing and corrupted files independently.
4. It uses ConcurrentLinkedQueue for safe result collection.
5. It allows performance comparison using different thread-pool sizes.
6. It avoids creating thousands of unnecessary threads.

The application can be improved further by using a database for permanent result storage and a persistent task queue for recovery after unexpected shutdowns.

11. Final Result

A concurrent and fault-tolerant student result-processing application was designed and implemented using ExecutorService, Callable, Future, CompletionService, and ConcurrentLinkedQueue.

The program successfully processes individual student files, records missing and corrupted files as failures, and continues processing the remaining files.

The small test dataset verifies three successful records and two failed records. The large dataset is designed to verify 9,980 successful files and 20 failed files.

The execution time is measured using thread pools containing 1, 2, 4, and 8 threads.

Thus, the application demonstrates concurrent processing, fault tolerance, safe result collection, and performance evaluation in Java.